

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 14 JANUARY 2020 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

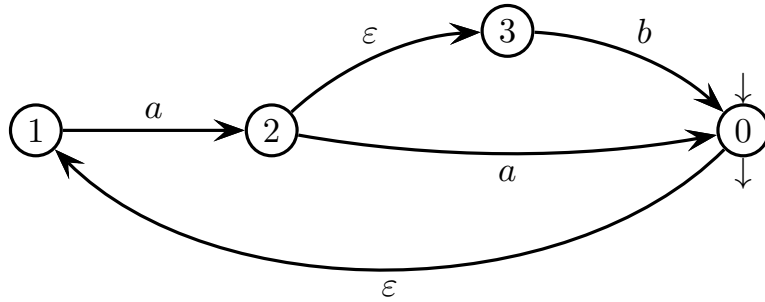
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Take a two-letter alphabet $\Sigma = \{a, b\}$. Consider the nondeterministic automaton A over Σ below (node 0 is both initial and final), which has spontaneous transitions (ε -transitions):



and consider also the regular expression R over Σ below:

$$R = (a a \mid a b \mid b a)^*$$

Answer the following questions:

- (a) Find all the valid strings of language $L(A)$ that have a length ≤ 4 . Do the same for language $L(R)$, too.
- (b) By using the back-propagation method, eliminate the spontaneous transitions of automaton A and obtain an equivalent deterministic automaton A_1 without spontaneous transitions. Clean and minimize automaton A_1 , if necessary. Test automaton A_1 with the valid short strings found at point (a) for automaton A .
- (c) By using the Berry-Sethi method, obtain a deterministic automaton A_2 equivalent to the regular expression R . Minimize automaton A_2 , if necessary. Test automaton A_2 with the valid short strings found at point (a) for expression R .
- (d) Say if language $L(R)$ is local or not, and shortly justify your answer.
- (e) (optional) By using a systematic method of your choice, obtain a deterministic automaton A_3 for the difference language $L_D = L(R) - L(A)$, i.e., $L(R) \cap \overline{L(A)}$, which contains the strings generated by R that are not recognized by A .
Then test automaton A_3 : by examining (only) the strings of length 4 found at point (a), find in alphabetical order the first two strings x and y such that $x \in L_D$ and $y \notin L_D$, and show that $x \in L(A_3)$ while $y \notin L(A_3)$.

2 Free Grammars and Pushdown Automata 20%

1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$. Then answer the following questions:
- (a) Write two *BNF* grammars G_1 and G_2 that generate the following two languages L_1 and L_2 , respectively:

$$L_1 = \{ w w^R \mid w \in \Sigma^* \}$$

$$L_2 = L(b^* a^* b^*)$$

- (b) Write a *BNF* nonambiguous grammar G_{12} that generates the intersection language $L_1 \cap L_2$. Argue that grammar G_{12} is not ambiguous.
- (c) Write a *BNF* grammar G_3 that generates the intersection language:

$$L_1 \cap L(b^* a^* b^* a^*)$$

Draw the syntax tree for a string (any string of your choice) of language $L(G_3)$.

- (d) (optional) Say if grammar G_3 is ambiguous, and suitably justify your answer.
-

2. We want to define a simplified language that describes the structure of certain flow graphs. The flow graph model to be considered is sketched below:

- A graph is a structure with a name, which is an identifier. A graph contains nodes and may contain arcs.
- A graph has exactly one input node and exactly one output node, both named and mandatory.
- A graph may have internal nodes. An internal node must have a name, which is an identifier.
- Two nodes may be connected by a (directed) arc, from the source node to the destination node. An arc may have a weight, which is a number.
- A graph may contain one or more subgraphs, which are graphs themselves, and so on at any containment depth.
- A subgraph, like a node, can be connected to nodes or subgraphs.
- The specification order of nodes and arcs is free, except that the input node precedes the output node. The subgraphs are listed after all the nodes and arcs.
- The lexical tokens, i.e., identifiers and numbers, are schematized by the terminals `id` and `num`, respectively, which do not need to be further expanded.
- For the language keywords and separators (and any other possibly unspecified details), refer to the sample description below as a guideline.

Here is a sample flow graph description:

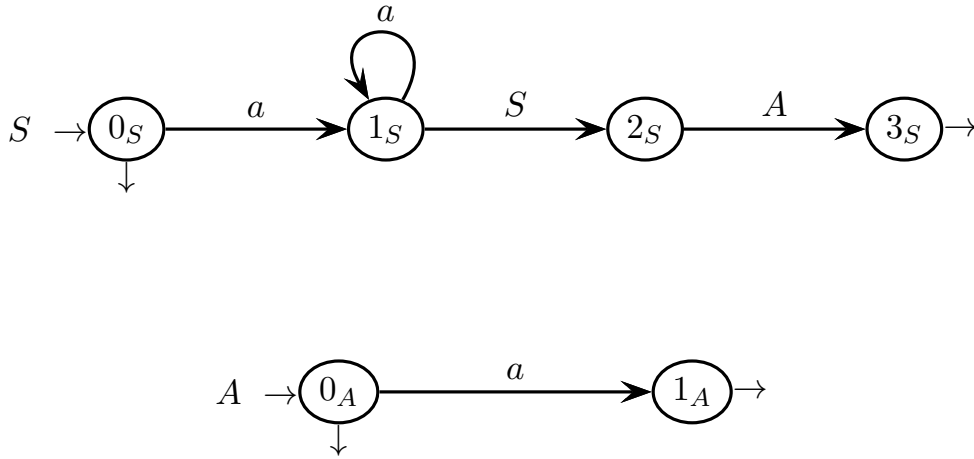
```
graph MAIN is
  input is alpha;
  node is beta;
  arc from alpha to beta weighted 1;
  output is gamma;
  arc from beta to SUB;
  arc from SUB to gamma weighted 2;
  subgraph SUB is
    ...
  endsubgraph
  ...
endgraph
```

Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the description language for the flow graph model sketched above.
- (b) Draw the (fragmentary) syntax tree of the sample graph description above.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, over the one-letter terminal alphabet $\Sigma = \{ a \}$ and the two-letter nonterminal alphabet $V = \{ A, S \}$ (axiom S):

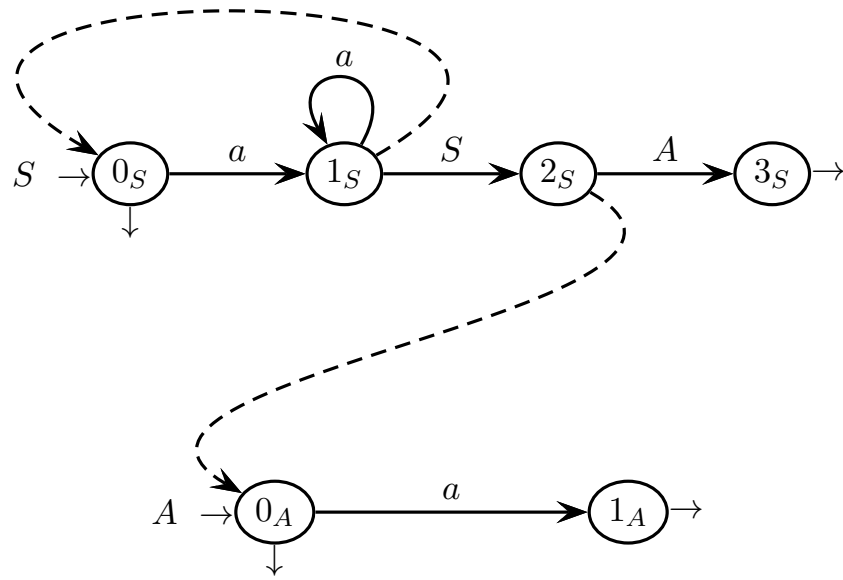


Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot automaton of grammar G . Determine whether grammar G is of type $ELR(1)$. Highlight all the conflicts that may be present in the pilot automaton, and indicate their type.
 - (b) By finding all the guide sets on the call arcs and exit arrows, determine whether grammar G is of type $ELL(1)$. Justify your answer.
 - (c) Draw the syntax tree (or all syntax trees if many exist) of the valid string aa .
 - (d) (optional) Analyze the valid string aa by means of the Earley method, and highlight the item(s) showing that the string is accepted.
-

please draw here the complete pilot automaton – question (a)

please write here the guide sets (on the call arcs and exit arrows) – question (b)



please draw here the syntax tree(s) of the valid string aa – question (c)

space for answering question (d)

Earley vector of string aa (to be filled in – the number of rows is not significant)

0	a	1	a	2

free space for continuation or correction if necessary – any questions

4 Language Translation and Semantic Analysis 20%

1. Consider the binary alphabet $\Sigma = \{0, 1\}$ and a syntactic translation function τ on bit strings of even length ≥ 0 . Translation τ works as follows:

$$\tau(\varepsilon) = \varepsilon$$

$$\tau(b_1 b_2 b_3 b_4 \dots b_{2n-2} b_{2n-1} b_{2n}) = b_1 b_3 \dots b_{2n-1} b_{2n} b_{2n-2} \dots b_4 b_2$$

with $b_i \in \Sigma$, where $1 \leq i \leq 2n$ and $n \geq 1$. In words, the destination string consists of the odd-position symbols of the source string, followed by the even-position ones, the latter in reverse order. Here is an example:

$$\tau(011100) = 010011$$

Take the following source grammar G_s (axiom A) for the source language of τ :

$$G_s \left\{ \begin{array}{l} A \rightarrow 0 B \\ A \rightarrow 1 B \\ A \rightarrow \varepsilon \\ B \rightarrow 0 A \\ B \rightarrow 1 A \end{array} \right.$$

Answer the following questions (use the spaces / tables on the next page):

- (a) Without changing the source grammar G_s , write a syntactic translation scheme (or grammar) G_τ that computes translation τ . Test your scheme G_τ by drawing the destination (or translation) tree of the example above.
- (b) Consider the inverse translation τ^{-1} , which is computed by the scheme $G_{\tau^{-1}}$ with switched source and destination grammars. Answer these points:
 - i. Is translation τ^{-1} a function on strings? Shortly justify your answer.
 - ii. Compute the translation $\tau^{-1}(011100)$.
 - iii. Characterize how translation τ^{-1} works on a generic bit string of even length, in the same style as shown above for translation τ . Namely, complete this:

$$\tau^{-1}(b_1 b_2 \dots b_n b_{n+1} \dots b_{2n-1} b_{2n}) = \text{(write output at page bottom)}$$

- (c) (optional) Say if translations τ and τ^{-1} are deterministic of *ELL* type, for $k = 1$. Shortly justify your answers.

To be completed with the output string – see question (b) point (iii):

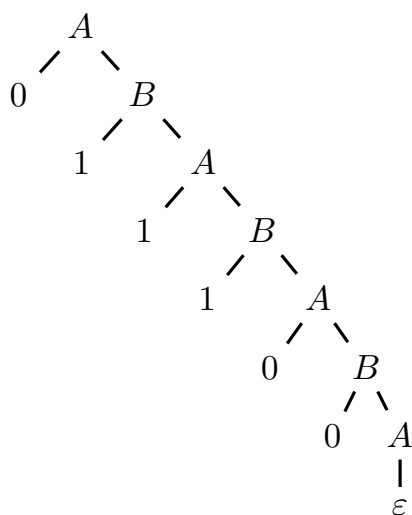
$$\tau^{-1}(b_1 b_2 \dots b_n b_{n+1} \dots b_{2n-1} b_{2n}) =$$

please write here the destination (or translation) grammar – question (a)

$$G_s \left\{ \begin{array}{l} \hline A \rightarrow 0 B \\ \hline A \rightarrow 1 B \\ \hline A \rightarrow \varepsilon \\ \hline B \rightarrow 0 A \\ \hline B \rightarrow 1 A \\ \hline \end{array} \right. \quad G_d \text{ or } G_\tau \left\{ \begin{array}{l} \hline \\ \hline \\ \hline \\ \hline \\ \hline \end{array} \right.$$

source tree

please draw here the destination
(or translation) tree – question (a)



space for answering questions (b-c) – see also the previous page (bottom)

continue elsewhere if necessary

2. We intend to compute a semantic translation τ of an arithmetic expression from the infix form into a partial postfix form, by using an attribute grammar. Below it is reported the syntax of the (infix) expression to be translated (axiom S), which is right-associative, where terminal a represents any variable or constant:

$$\left\{ \begin{array}{l} 1: S \rightarrow E \\ 2: E \rightarrow T + E \\ 3: E \rightarrow T \\ 4: T \rightarrow (E) \\ 5: T \rightarrow a \end{array} \right.$$

Translation τ keeps all the parentheses as they are and works on the rest as follows:

- it leaves in *infix form* every (sub)expression (immediately nested in a parenthesis pair or at the outermost level) that has an *odd* number of terms (1, 3, ...)
- it puts in *postfix form* every (sub)expression (immediately nested in a parenthesis pair or at the outermost level) that has an *even* number of terms (2, 4, ...)

Here are two sample translations of increasing complexity:

$$\tau \left(\overbrace{a}^o + \left(\overbrace{a}^o + \overbrace{a}^e \right) + \overbrace{a}^o \right) = a +_1 (a a +_2) +_3 a$$

$$\tau \left(a +_1 \left(a +_2 \left(a +_3 a \right) +_4 a \right) \right) = a \left(a +_2 \left(a a +_3 \right) +_4 a \right) +_1$$

The operator “+” is numbered to visualize how translation τ works. The position of each term is tagged odd or even (o or e) in each (sub)expression (top sample only) for the same purpose. Such annotations are not part of the attribute grammar.

The translated expression is represented by an attribute τ of type string in the root node of the syntax tree. On the next page, there is a list of grammar attributes (including τ) to use. Do not add or modify any attribute. There is also a decorated tree that illustrates how to use such attributes to translate a string (bottom sample).

Answer the following questions (use the tables / drawings / spaces on the next pages):

- Write the semantic functions for computing attribute *odd*.
- Write the semantic functions for computing attribute *totOdd*.
- Write the semantic functions for computing attribute τ .
- Write the translation of this expression (different from the two samples above):

$$(a + a + a) + a$$

and decorate its tree with all the appropriate attribute values in each node.

- (optional) Draw the dependency graph for each grammar rule.

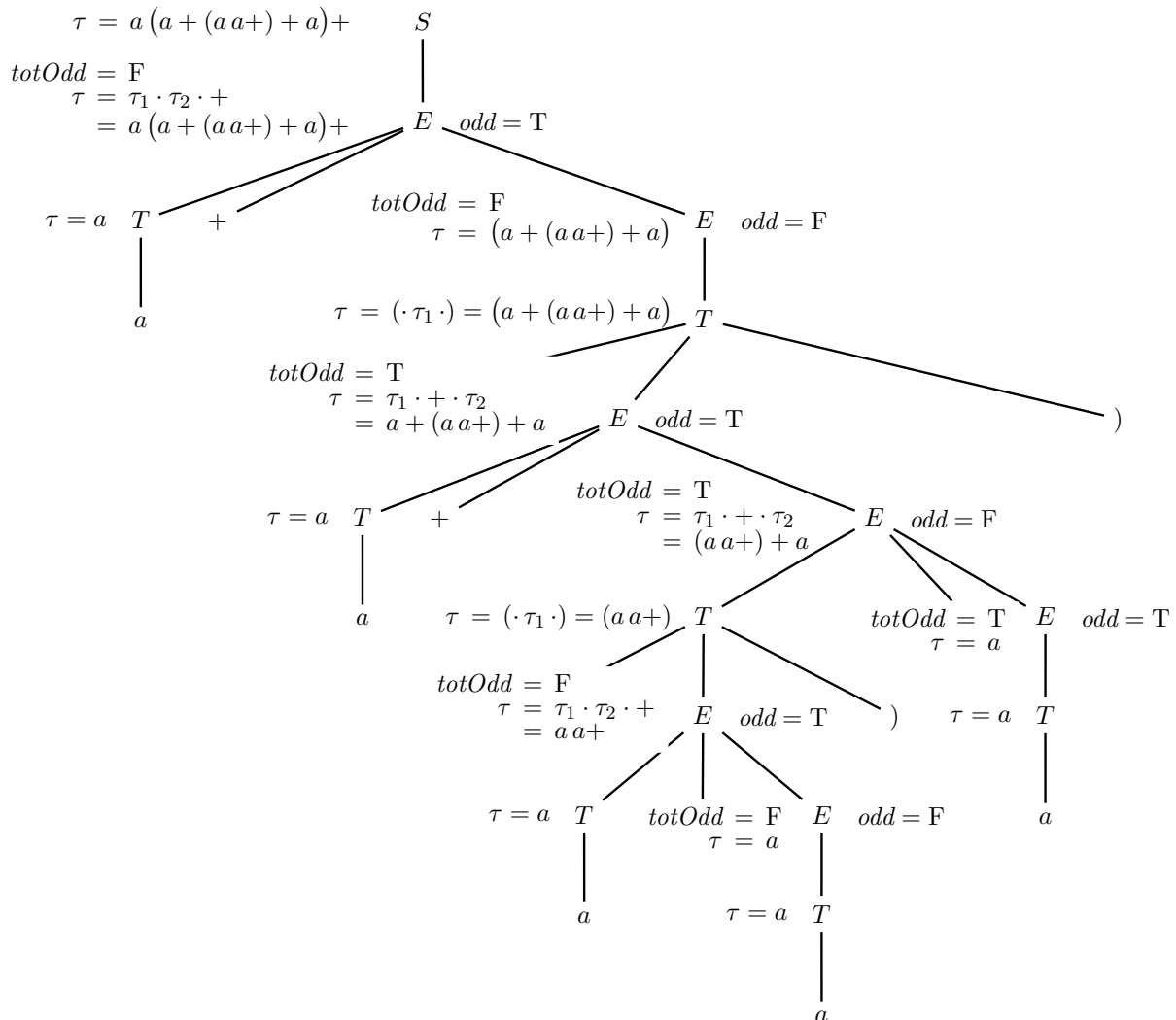
attribute list and sample decorated tree illustrating the attribute use

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
<i>odd</i>	right	boolean	<i>E</i>	true if the node occupies an odd position (first, third, etc) in the current sequence of nodes of type <i>E</i>
<i>totOdd</i>	left	boolean	<i>E</i>	true if the total number of nodes of type <i>E</i> in the current sequence is odd
τ	left	string	T, E, S	in the root, the translation of the entire tree; in the other nodes, suitable fragments of the overall translation

How to use the attributes to translate the bottom sample string:

$$\tau\left(a+(a+(a+a))+a\right)=a\left(a+(aa+)+a\right)+$$

is shown by this tree (some leaves are masked by the attributes):



tables for answering questions (a-b-c)
only the rules relevant for each attribute are reported

#	<i>syntax</i>	<i>semantics of the <u>right</u> attribute “odd” – question (a)</i>
1:	$S_0 \rightarrow E_1$	
2:	$E_0 \rightarrow T_1 + E_2$	
4:	$T_0 \rightarrow (E_1)$	

#	<i>syntax</i>	<i>semantics of the <u>left</u> attribute “totOdd” – question (b)</i>
2:	$E_0 \rightarrow T_1 + E_2$	
3:	$E_0 \rightarrow T_1$	

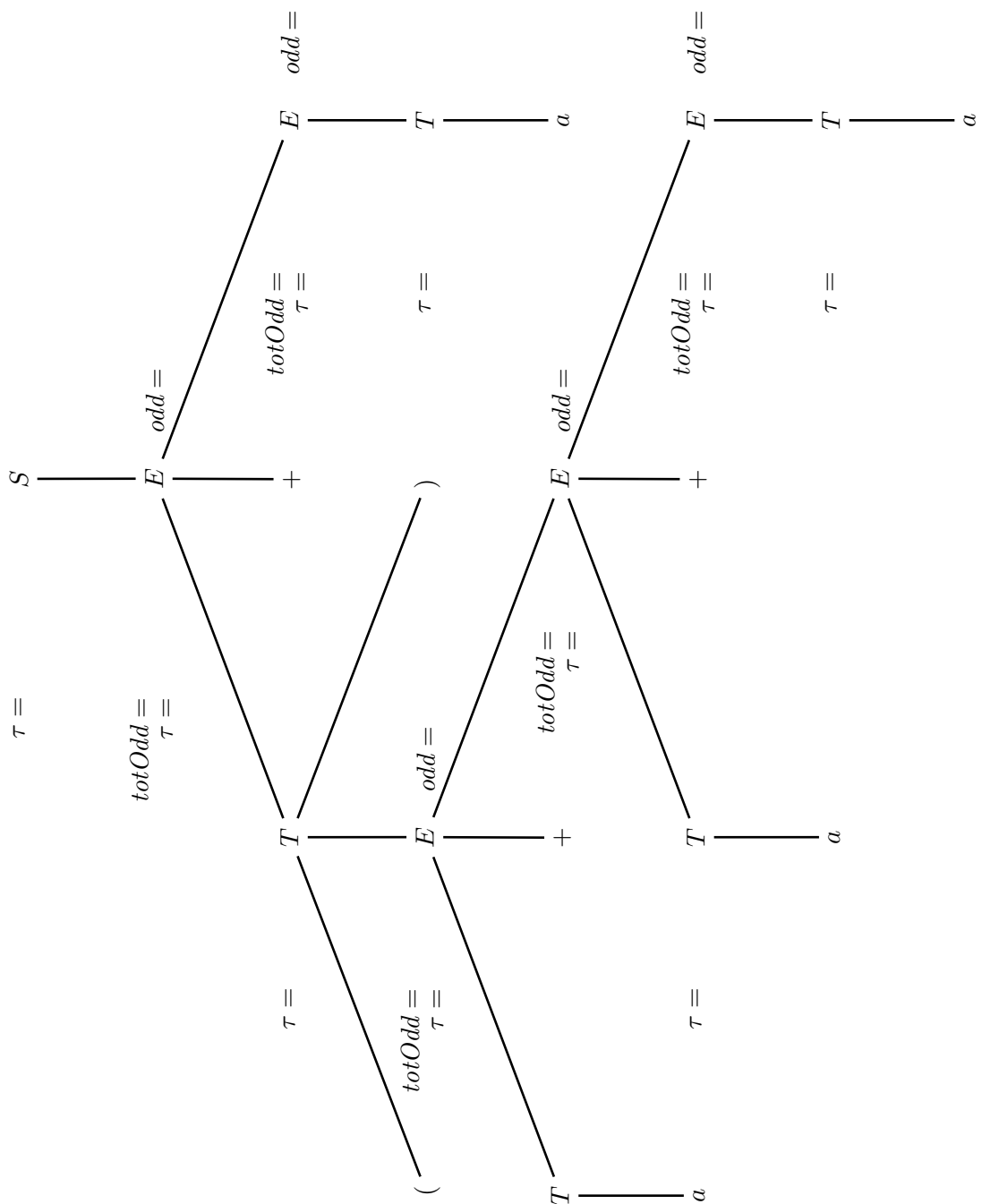
#	<i>syntax</i>	<i>semantics of the <u>left</u> attribute “τ” – question (c)</i>
1:	$S_0 \rightarrow E_1$	
2:	$E_0 \rightarrow T_1 + E_2$	
3:	$E_0 \rightarrow T_1$	
4:	$T_0 \rightarrow (E_1)$	
5:	$T_0 \rightarrow a$	

space and tree for answering question (d)

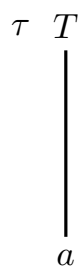
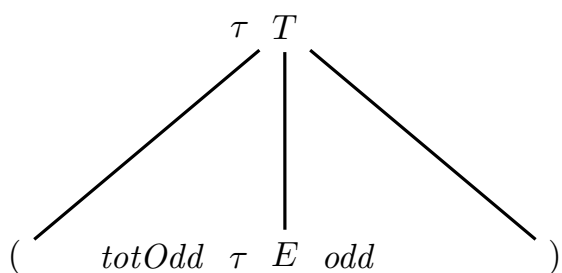
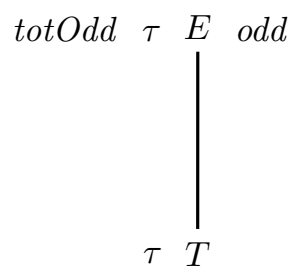
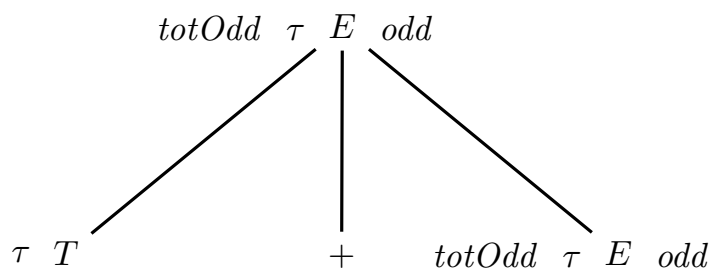
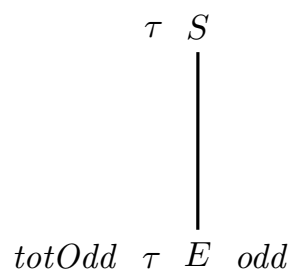
Write here the translation of the proposed string “ $(a + a + a) + a$ ”:

$$\tau \left((a + a + a) + a \right) =$$

Decorate with all the attribute values the syntax tree of such a string:



elementary grammar trees for answering question (e)
draw here the dependency graph of each grammar rule



free space for continuation or correction if necessary – any questions

free space for continuation or correction if necessary – any questions